

## Backend Strategy

You'll use:

- Node.js + Express
- MongoDB (Mongoose)

👉 Even if you don't build backend now, design it properly.

---

### 1. Core Database Design

#### 1. Project Schema (MOST IMPORTANT)

```
const mongoose = require("mongoose");

const projectSchema = new mongoose.Schema({
  title: { type: String, required: true },

  shortDescription: { type: String, required: true },
  fullDescription: { type: String },

  techStack: [{ type: String }],

  category: {
    type: String,
    enum: ["Systems", "AI", "FullStack"],
    required: true
  },

  githubLink: { type: String },
  demoLink: { type: String },

  images: [{ type: String }],

  features: [{ type: String }],
  challenges: [{ type: String }],
  learnings: [{ type: String }],

  isFeatured: { type: Boolean, default: false },
```

```
    createdAt: { type: Date, default: Date.now }  
  });
```

```
module.exports = mongoose.model("Project", projectSchema);
```

---

## ● 2. User/Admin Schema

👉 Needed for future admin panel

```
const userSchema = new mongoose.Schema({  
  name: String,  
  email: { type: String, unique: true },  
  
  password: String,  
  
  role: {  
    type: String,  
    enum: ["admin"],  
    default: "admin"  
  }  
});
```

```
module.exports = mongoose.model("User", userSchema);
```

---

## ● 3. Contact Messages Schema

```
const contactSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  message: String,  
  
  createdAt: {  
    type: Date,  
    default: Date.now  
  }  
});
```

```
module.exports = mongoose.model("Contact", contactSchema);
```

---

#### 4. Skills Schema

```
const skillsSchema = new mongoose.Schema({  
  category: String, // e.g., "Backend", "Systems"
```

```
  items: [String]  
});
```

```
module.exports = mongoose.model("Skills", skillsSchema);
```

---

#### 2. API Design (REST)

##### Project APIs

```
GET  /api/projects    → Get all projects  
GET  /api/projects/:id → Get single project  
POST /api/projects    → Create project (admin)  
PUT  /api/projects/:id → Update project  
DELETE /api/projects/:id → Delete project
```

---

##### Auth APIs

```
POST /api/auth/login  
POST /api/auth/register (optional)
```

---

##### Contact API

```
POST /api/contact  
GET  /api/contact (admin)
```

---

#### 3. Authentication Flow

Use:

- JWT Authentication

Flow:

Login → Server verifies → returns JWT

→ Store in frontend

→ Send token in headers for protected routes

---

#### 4. Backend Folder Structure

backend/

controllers/

projectController.js

authController.js

contactController.js

models/

Project.js

User.js

Contact.js

Skills.js

routes/

projectRoutes.js

authRoutes.js

contactRoutes.js

middleware/

authMiddleware.js

config/

db.js

server.js

---

#### 5. Data Flow (Important)

Frontend (Next.js)



API (Express)



MongoDB

---

## 6. Optimization for YOUR Projects

Because your projects are **complex**, extend schema:

**Add this:**

```
architecture: String,  
demoVideo: String,  
complexityLevel: {  
  type: String,  
  enum: ["Low", "Medium", "High"]  
}
```

---

## 7. Advanced Features (Future Upgrade)

You can later add:

### ◆ GitHub Sync

Auto-fetch repos via API

### ◆ Analytics

Track:

- project clicks
- recruiter behavior

### ◆ Blog System

MDX or DB-based posts

---

## 8. Reality Check

For now:

-  You do NOT need backend immediately
-  Start with static JSON

 Add backend only when:

- you need admin panel
  - or dynamic updates
-